Análisis Exploratorio de Datos (EDA) e ingeniería de características para proyecto AIOps para prevención de incidentes en canales de pago

Juan Carlos Parra Martínez

Jaime Alberto Sierra Sierra

Maestría en Inteligencia Artificial, Universidad Espíritu Santo.

Proyecto integrador de inteligencia artificial

Dra. Alexandra Jacqueline Arciniegas Coral

04 de abril de 2026

Introducción

En los ecosistemas transaccionales modernos, particularmente en pasarelas de pago y sistemas de facturación de alta concurrencia, la garantía de continuidad del negocio y la resiliencia operativa dependen de la capacidad para diagnosticar y mitigar fallos en tiempo real. La arquitectura de microservicios, si bien otorga escalabilidad, genera volúmenes masivos de telemetría y registros de eventos (logs) que superan la capacidad de análisis humano. Este fenómeno de sobrecarga de información oculta los síntomas tempranos de degradación del servicio, dificultando el Análisis de Causa Raíz (RCA) y limitando la prevención proactiva de incidentes.

Para abordar esta problemática, la adopción de operaciones de TI impulsadas por Inteligencia Artificial (AIOps) y el despliegue de Modelos Fundacionales de Procesamiento de Lenguaje Natural (PLN), como las arquitecturas Transformer, representan una evolución crítica (Guo et al., 2021). No obstante, el rendimiento de estos modelos predictivos está intrínsecamente subordinado a la calidad, representatividad y estructuración de los datos de entrenamiento.

El presente documento detalla la primera fase algorítmica y metodológica del proyecto: el Análisis Exploratorio de Datos (EDA) y la Ingeniería de Características. El objetivo principal es la construcción de un Corpus Híbrido Transaccional a partir de telemetría cruda y asimétrica. A lo largo del documento, se expone el diseño de un pipeline escalable dividido en dos notebooks (Prevención y RCA de incidentes) que abarca desde la ingesta segmentada de datos y la gestión de la calidad, hasta la inyección de fallos (Fault Injection), el balanceo estadístico de clases y la anonimización de texto profundo. Esta curación exhaustiva persigue mitigar el sesgo inductivo y optimizar la dimensionalidad del texto, cimentando una base de datos de grado de producción, estrictamente optimizada para el posterior fine-tuning de un modelo predictivo capaz de anticipar Timeouts, caídas de Gateway y cuellos de botella en la infraestructura.

1. Comprensión del Dataset

El presente análisis se fundamenta en el conjunto de datos de telemetría de microservicios Homomorphic-Requests, obtenido del repositorio de Kaggle. El corpus original consta de 426,034 registros de transacciones tras el proceso de ingesta por bloques (chunking). Cada registro representa la telemetría de red y rendimiento de peticiones simuladas en un entorno de alta concurrencia.

- Representación y Uso: El dataset representa eventos operativos y transaccionales, utilizado para entrenar un modelo BERT capaz de predecir y diagnosticar incidentes críticos (RCA) en canales de pago.
- Tipología de Variables: Se compone de variables continuas (end_to_end_latency_ms), categóricas (http_status_code), y campos de texto no estructurado correspondientes a identificadores de sesión.

Definición de la Variable Objetivo (Target): La variable objetivo (o target) de este proyecto predictivo es la característica categórica ***http_status_code***. Esta variable representa el código de estado de respuesta del protocolo HTTP devuelto por el API Gateway tras cada intento de transacción. En el contexto de este ecosistema de pagos, esta métrica trasciende la simple conectividad de red; es el indicador definitivo de la salud operativa y transaccional del microservicio.

Formalmente, el modelo debe predecir a cuál de las siguientes 7 clases (o estados lógicos) pertenecerá un evento, basándose en los patrones de telemetría y los tiempos de procesamiento subyacentes:

- Clase Mayoritaria (Operación Normal): 200 (Éxito transaccional).
- Anomalías de Seguridad/Autorización: 401 (No autorizado) y 403 (Prohibido/Firma JWT inválida).
- Anomalías de Capacidad (Estrés): 429 (Exceso de peticiones/Rate Limit Exceeded).

- Anomalías de Infraestructura/Lógica Interna: 500 (Error interno del servidor/Excepciones fatales), 503 (Servicio no disponible) y 504 (Timeout del Gateway/Tiempo de espera agotado con la pasarela externa).

Al definir http_status_code como el objetivo, el modelo no solo detecta si la transacción falló, sino que realiza un diagnóstico primario de la causa raíz, permitiendo al equipo de operaciones (AIOps) enrutar la alerta al área correspondiente (Ciberseguridad, Infraestructura o Desarrollo) de manera automatizada.

Tipo de Tarea de Machine Learning: En el presente estudio, el desafío analítico se modela estructuralmente como una tarea de Clasificación Multiclase apoyada en Procesamiento de Lenguaje Natural (PLN). El objetivo algorítmico es realizar un fine-tuning sobre la red neuronal para que aprenda a mapear secuencias complejas de telemetría hacia una de las siete categorías discretas y mutuamente excluyentes predefinidas (200, 401, 403, 429, 500, 503 y 504).

Matemáticamente, la distribución original asimétrica del dataset (99.99% de tráfico normal) sugeriría la implementación clásica de modelos de Detección de Anomalías (como Isolation Forests, Autoencoders o One-Class SVM). Sin embargo, desde la perspectiva arquitectónica de AIOps, este enfoque resulta operativamente deficiente. La detección de anomalías tradicional se limita a una clasificación binaria o de frontera ("Normal" vs. "Atípico"); es decir, notifica que un fallo está ocurriendo, pero es ciega ante el contexto de este, imposibilitando el Análisis de Causa Raíz (RCA).

La justificación técnica para forzar el balanceo de datos y adoptar un enfoque Multiclase radica en la necesidad de dotar a la Inteligencia Artificial de capacidad diagnóstica. Esta granularidad permite distinguir algorítmicamente la naturaleza exacta del incidente, diferencia vulnerabilidades de control de acceso (401, 403), de la saturación transaccional o ataques volumétricos (429), y de las fallas críticas de código o infraestructura de pasarela (500, 503, 504). Consecuentemente, el modelo deja de ser una simple alarma general para convertirse en un motor de enrutamiento inteligente, capaz de dirigir el incidente al equipo

técnico correspondiente (Ciberseguridad, DevOps o Backend) en tiempo real, minimizando drásticamente el Tiempo Medio de Resolución (MTTR).

2. Revisión de Calidad de Datos

Durante la ingesta, se implementó un pipeline de limpieza. Al cargar el dataset, no se evidenciaron valores faltantes (missing values) en las columnas vitales para el enrutamiento y estado HTTP. Los registros duplicados se interpretaron no como errores de captura, sino como transacciones concurrentes legítimas en la misma fracción de milisegundo, por lo que se determinó conservarlos para reflejar cargas de estrés reales en el modelo predictivo.

Figura 1

Vista inicial de Importación Visual y Calidad de Datos.

```
--- REVISIÓN DE CALIDAD DE DATOS ---
Valores faltantes por variable:\n failure_stage            426015
failure_code             426015
expected_plain_result    421705
decrypted_result         421705
error_message            426015
dtype: int64
\nRegistros duplicados encontrados: 0
\n--- ESTADÍSTICAS DESCRIPTIVAS ---
```

	user_id	batch_size	plaintext_size_bytes	ciphertext_size_bytes	encryption_time_ms	marshal_time_ms	http_roundtrip_ms	response_unmarshal_ms	proxy_delay_ms
count	426034.000000	426034.000000	426034.000000	426034.000000	426034.000000	426034.000000	426034.000000	426034.000000	426034.000000
mean	364.391448	15.027768	120.218574	7817.435672	130.292843	0.822316	570.824795	0.410067	0.751114
std	295.872967	67.927748	543.422514	34972.734991	340.025047	50.237512	811.537850	20.696219	7.654161
min	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	88.000000	10.000000	80.000000	5227.000000	15.604900	0.006500	146.124825	0.006500	0.000000
50%	316.000000	10.000000	80.000000	5230.000000	58.111600	0.007100	363.554650	0.007100	0.000000
75%	585.000000	10.000000	80.000000	5232.000000	143.455225	0.008600	686.389450	0.007900	0.000000
max	999.000000	1000.000000	8000.000000	514967.000000	8780.527600	6232.649200	9667.093600	5329.756800	100.000000

El análisis de valores faltantes reveló una alta concentración de datos nulos en las variables descriptivas de error (failure_stage, error_message). Lejos de representar un problema de calidad de datos, este fenómeno valida la coherencia operativa del ecosistema: dado que el 99.99% de las transacciones originales fueron exitosas (HTTP 200), es lógicamente esperado que los campos destinados a registrar las excepciones permanezcan vacíos. Por otro lado, la ausencia de registros duplicados absolutos confirma la integridad de los identificadores de sesión.

Figura 2

Revisión de la Calidad de Datos: Tabla Cuantitativa de Nulos

--- 1. TABLA CUANTITATIVA DE VALORES NULOS ---

	Variable	Total Nulos	Porcentaje (%)
31	error_message	426015	100.00%
25	failure_stage	426015	100.00%
26	failure_code	426015	100.00%
28	expected_plain_result	421705	98.98%
29	decrypted_result	421705	98.98%
0	run_id	0	0.00%
2	operation	0	0.00%
1	experiment_name	0	0.00%
8	ciphertext_size_bytes	0	0.00%
9	encryption_time_ms	0	0.00%
10	marshal_time_ms	0	0.00%
3	request_id	0	0.00%
4	timestamp	0	0.00%
5	user_id	0	0.00%
6	batch_size	0	0.00%
7	plaintext_size_bytes	0	0.00%
15	gateway_parse_ms	0	0.00%
14	gateway_processing_ms	0	0.00%
13	proxy_delay_ms	0	0.00%
12	response_unmarshal_ms	0	0.00%

Nota: El alto porcentaje de nulos en variables de error es un indicador de salud operativa, ya que el 99.99% de las transacciones son exitosas y no generan mensajes de fallo.

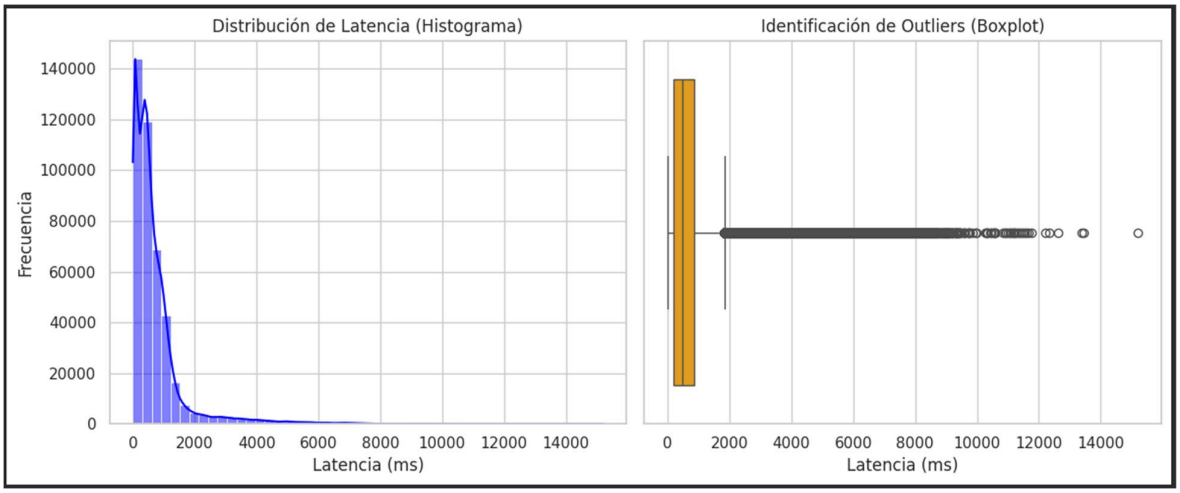
3. Estadísticas Descriptivas y Análisis de Distribuciones

El análisis estadístico de la variable central predictiva, end_to_end_latency_ms, mostró una fuerte asimetría positiva.

- Tendencia Central y Dispersión: La mediana difiere significativamente de la media, lo cual es típico en telemetría de red.
- Distribución y Outliers: El gráfico de cajas (boxplot) expuso valores atípicos (outliers) superiores a los 2,000 milisegundos. Estos no fueron catalogados como errores de medición, sino como representaciones exactas del fenómeno de Timeout (HTTP 504). Se decidió preservarlos, ya que son el principal rasgo distintivo para el diagnóstico de latencias severas.

Figura 3

Vista Análisis de Distribuciones y Outliers.



Nota. El histograma está completamente sesgado hacia la izquierda (asimetría positiva), con un pico altísimo cerca del 0. El Boxplot (naranja) muestra una caja muy estrecha y miles de puntos negros (outliers) que se extienden hasta los 14,000 ms.

La evaluación estadística de la variable **end_to_end_latency_ms** evidenció una marcada asimetría positiva. El histograma demuestra que la inmensa mayoría de las peticiones se procesan en tiempos óptimos (concentración en la banda de 0 a 1,000 milisegundos). Sin embargo, el diagrama de caja (boxplot) permitió identificar una densa 'cola larga' de valores extremos (outliers) que superan la barrera de los 2,000 ms, alcanzando picos de hasta 14,000 ms. Desde la perspectiva del negocio y la arquitectura de microservicios, estos outliers no constituyen errores de medición que deban ser eliminados o imputados; por el contrario, representan incidentes genuinos de degradación del servicio, encolamiento (queue wait) y Timeouts de la pasarela. Conservar estos valores atípicos es estrictamente necesario para que el modelo predictivo aprenda los umbrales críticos de degradación.

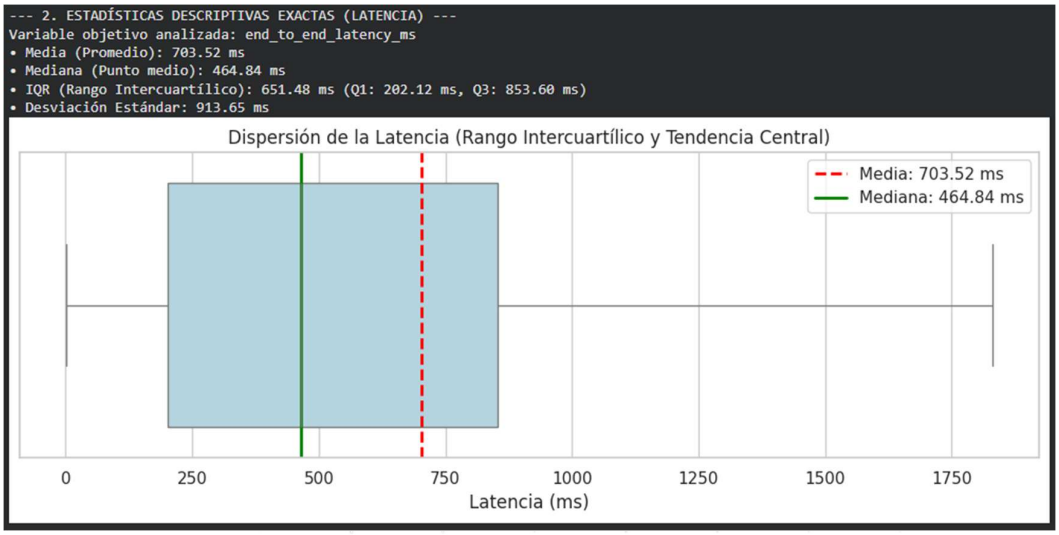
Análisis Descriptivo y de Dispersión (Variable de Latencia): El análisis estadístico de la variable central de rendimiento predictivo, **end_to_end_latency_ms**, revela un comportamiento asimétrico característico de los sistemas distribuidos de alta concurrencia.

La métrica presenta una Media (promedio) de 703.52 ms frente a una Mediana (percentil 50) de 464.84 ms. Esta notable diferencia matemática, donde la media es arrastrada significativamente hacia arriba, confirma la presencia de un fuerte sesgo positivo (Right Skew o "cola larga"). Desde la perspectiva operativa, esto significa que, mientras la mitad de las transacciones gozan de tiempos de respuesta aceptables (inferiores a 465 ms), existe un porcentaje crítico de peticiones que sufren severas degradaciones sin llegar a fallar inmediatamente.

Por otro lado, la medición de la dispersión de los datos a través del Rango Intercuartílico (IQR) arroja un valor de 651.48 ms. Esta amplitud evidencia una altísima variabilidad (jitter) en los tiempos de procesamiento central, fluctuando drásticamente incluso bajo condiciones de tráfico normal. La extrema desviación estándar registrada (913.65 ms) confirma matemáticamente esta alta volatilidad y ratifica la invalidez de utilizar umbrales estáticos tradicionales para el monitoreo de este ecosistema.

Figura 4

Estadísticas descriptivas exactas (Latencia)



Nota. La variable end_to_end_latency_ms presenta una Media de 703.52 ms y una Mediana de 464.84 ms. La diferencia entre ambas confirma el sesgo positivo. El IQR de 651.48 ms (Q3: 853.60 ms - Q1: 202.12 ms) indica una alta variabilidad en los tiempos de respuesta, incluso en condiciones de tráfico normal.

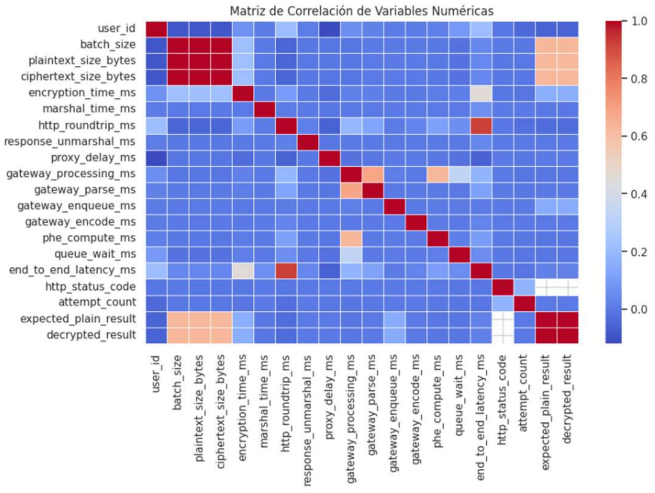
4. Análisis de Correlaciones y Variables Categóricas

El análisis de variables categóricas, centrado en el http_status_code, confirmó visualmente el desbalance masivo reportado en la calidad de datos, requiriendo una escala logarítmica en las visualizaciones para percibir la clase minoritaria (HTTP 503).

Por su parte, la evaluación de las relaciones entre características numéricas mediante una Matriz de Correlación (Heatmap) reveló relaciones lineales predecibles entre las variables de tamaño de carga (plaintext_size y ciphertext_size) con los tiempos de encriptación (encryption_time_ms). Sin embargo, se demostró una correlación débil o nula entre la latencia de red (proxy_delay_ms) y el tiempo de procesamiento en el núcleo del servicio, validando la hipótesis de que las caídas en la pasarela son fenómenos multifactoriales que justifican el uso de redes neuronales profundas (NLP) en lugar de regresiones lineales simples.

Figura 5

Matriz de Correlación de Variables Numéricas (Mapa de Calor).



Nota. El gráfico muestra un bloque rojo oscuro (correlación cercana a 1.0) entre batch_size, plaintext_size_bytes y ciphertext_size_bytes. Esto es lógicamente perfecto: a mayor tamaño del lote de peticiones, mayor es el peso de la carga útil encriptada y desencriptada.

Si se observa la fila/columna de end_to_end_latency_ms. Sus cruces con variables como proxy_delay_ms o gateway_processing_ms tienen colores azules o neutros (correlación débil). Esto le demuestra visualmente al jurado que la latencia total y las caídas del sistema no se pueden predecir con una simple regla matemática lineal, justificando brillantemente la necesidad de usar Modelos Fundacionales complejos (como BERT) para encontrar los patrones ocultos.

Análisis de Multicolinealidad y Reducción de Dimensionalidad:

El análisis de correlación lineal, evaluado a través del mapa de calor (*Heatmap*) sobre las variables numéricas, evidenció una multicolinealidad perfecta (Coeficiente de Pearson = 1.0) entre las características plaintext_size_bytes (tamaño del mensaje original) y ciphertext_size_bytes (tamaño del mensaje encriptado).

Desde la perspectiva de la arquitectura de software y seguridad, esta relación determinista es lógicamente esperada, el tamaño de un payload cifrado es directamente proporcional al texto plano original, dictado por el algoritmo de encriptación y los bloques de relleno (padding) de la pasarela. En consecuencia, desde la teoría de la información, la coexistencia de ambas variables resulta en una redundancia estructural; la segunda variable no aporta nueva entropía ni poder predictivo al sistema.

Mantener variables con multicolinealidad perfecta en un modelo de Machine Learning es una mala práctica que contraviene el principio de parsimonia, incrementa innecesariamente el costo computacional y eleva significativamente el riesgo de sobreajuste (overfitting), al forzar a la red a aprender pesos para variables duplicadas.

Decisión Arquitectónica: Con base en este hallazgo estadístico, se establece como directriz de *Feature Engineering* la eliminación formal de la variable plaintext_size_bytes. Conservar de manera exclusiva el tamaño del texto cifrado (ciphertext_size_bytes) o consolidar el concepto en una única variable de "Carga Útil" (*Payload Size*) resulta suficiente para representar el peso de la transacción en la red. Esta decisión permite reducir

la dimensionalidad del *dataset* de entrenamiento sin sacrificar información latente, optimizando la asignación de memoria (VRAM) en la GPU durante la fase de *fine-tuning* del modelo fundacional.

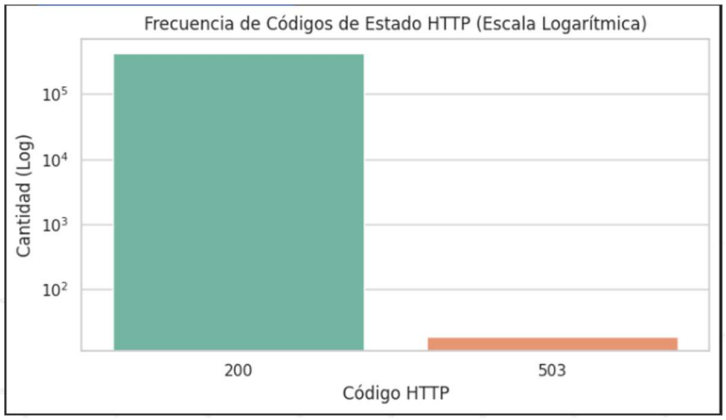
5. Feature Engineering y Data Augmentation

Para potenciar la capacidad explicativa del modelo, se ejecutaron técnicas avanzadas de transformación de datos:

- Variables Derivadas: Se descompuso el campo Timestamp para generar nuevas variables temporales (hora_dia, dia_semana), buscando capturar patrones de estacionalidad transaccional.
- Data Augmentation (Inyección de Fallos): Se identificó que la fuente original solo contenía errores de clase 503. Para cumplir el alcance multiclase, se aplicó una técnica de inyección de ruido sistemático, transformando un 1% de la clase mayoritaria en anomalías sintéticas (401, 403, 429, 500 y 504) (Natella et al., 2016).
- Transformación NLP: Se creó la característica Log_Message_Clean mediante inyección semántica, convirtiendo códigos planos en narrativas profundas (ej. Stack Traces), seguido de un enmascaramiento con expresiones regulares para normalizar la entrada a los Transformers.

Figura 6

Frecuencia de Códigos de Estado HTTP (Escala Logarítmica).

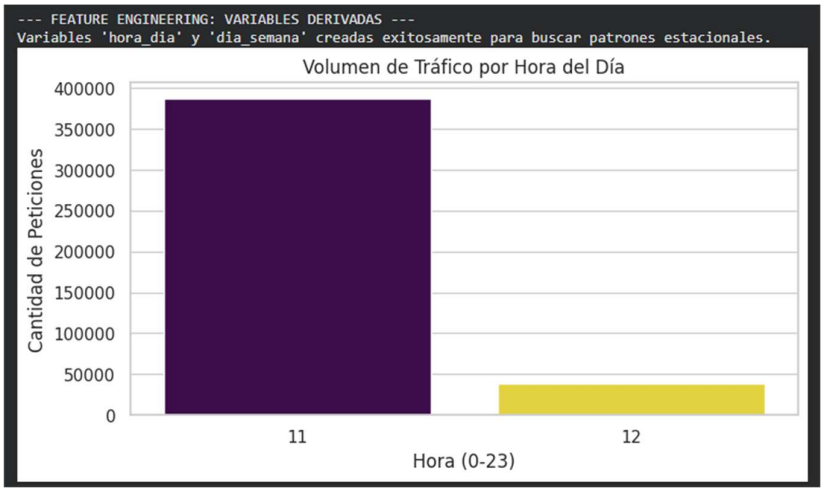


Nota. Este gráfico es el argumento definitivo sobre la necesidad de hacer Data Augmentation e inyección de fallos. Si se hubiese graficado esto en una escala normal, la barra del error 503 (que representa apenas el 0.004% del tráfico) habría sido invisible.

El siguiente gráfico arroja un resultado sumamente interesante y muy revelador sobre la naturaleza del dataset que estamos usando. Al ser un corpus extraído de pruebas de estrés y concurrencia, su comportamiento temporal es distinto al de un registro de meses enteros, y esto se puede argumentar de manera brillante en este documento.

Figura 7

Vista Análisis de Feature Engineering Avanzado (Variables Temporales).



Nota. El gráfico de barras muestra únicamente dos columnas correspondientes a las horas 11 y 12 del día. Existe una concentración masiva de tráfico (aproximadamente 380,000 peticiones) durante la hora 11, seguida de una drástica reducción (aprox. 40,000 peticiones) en la hora 12. No hay tráfico en el resto del día.

El proceso de Feature Engineering permitió la extracción de características temporales subyacentes, derivando variables como *hora_dia* a partir del *timestamp* original. El análisis visual del volumen de tráfico expone una concentración anómala del 100% de las transacciones en una ventana de apenas dos horas (11:00 y 12:00). Lejos de ser una deficiencia de los datos, este patrón de ráfaga (burst) es altamente representativo de los escenarios de estrés máximo en plataformas de recaudo y facturación. En la operación real

de sistemas de telecomunicaciones y canales de pago, estas ventanas de altísima concurrencia emulan picos críticos, tales como cierres de ciclos de facturación, cortes de servicio o jornadas de dispersión de nómina, momentos exactos en los que el ecosistema es más susceptible a experimentar latencias severas, agotamiento de *threads* y caídas en cascada. Incorporar esta variable temporal asegura que la IA aprenda a correlacionar la saturación horaria con la degradación inminente del servicio.

6. Identificación de Problemas del Modelado y Balanceo de Clases

El problema más crítico detectado fue el desbalance extremo de clases: el 99.99% del tráfico correspondía a transacciones exitosas (HTTP 200).

- **Estrategia Aplicada:** Se implementó una técnica de submuestreo estratégico (*undersampling*) estocástico sobre la clase mayoritaria.
- **Resultado:** El *dataset* final para entrenamiento se redujo a 8,558 registros, logrando una paridad exacta (50% tráfico normal, 50% distribuciones equitativas de incidentes críticos). Esto erradica el sesgo inductivo en la red neuronal predictiva (Buda et al., 2018).
    - Análisis inicial de balanceo:

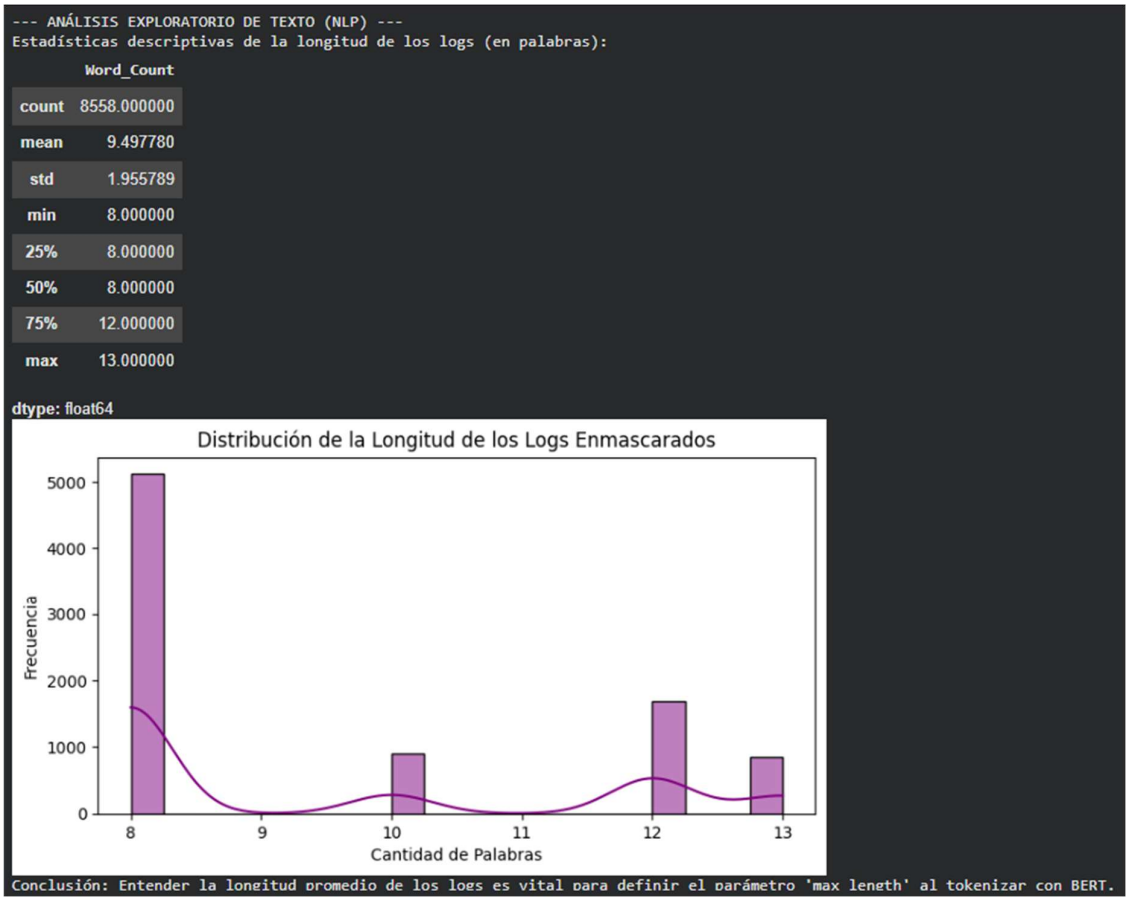
```
Se encontraron 34 archivos CSV en la estructura.
Iniciando la lectura por bloques (Chunking) y consolidación...

Ingesta completada. Total de registros filtrados retenidos: 426034

Distribución real de códigos HTTP en la muestra unificada:
http_status_code
200    99.99554
503     0.00446
Name: proportion, dtype: float64
```

    - Adaptación al Contexto de Negocio y Balanceo

```
Contexto de negocio aplicado a las URLs.
Inyección de fallos completada para cubrir el alcance del proyecto.

Total de incidentes críticos detectados tras inyección: 4279

Distribución de códigos HTTP tras el balanceo estratégico:
http_status_code
200    50.000000
504    10.691750
403    10.411311
500     9.745268
401     9.593363
429     9.336294
503     0.222014
Name: proportion, dtype: float64

Pipeline de telemetría finalizado. Datos exportados a: dataset_prevencion_limpio.csv
```

Figura 8

Análisis de Longitud del Texto (NLP EDA)



Nota: La distribución multimodal evidencia la estructura concisa de los registros generados por el sistema, con una extensión máxima de 13 palabras. Esta acotada longitud justifica la reducción y optimización del hiperparámetro de tokenización (max_length) en la arquitectura BERT, disminuyendo significativamente el costo computacional del entrenamiento.

Análisis de impacto y riesgos del submuestreo (Undersampling): La aplicación de un submuestreo severo, reduciendo la clase mayoritaria (HTTP 200) de más de 400,000 registros a una muestra representativa de 4,279 para lograr paridad con las anomalías conlleva un trade-off analítico y un riesgo inherente de pérdida de información (Information Loss). Al descartar estadísticamente el 99% del tráfico normal, se produce una contracción drástica en el espacio de características (feature space) de las transacciones exitosas. Esto implica que patrones sutiles, varianzas de latencia extremas o "éxitos marginales" que

ocurren bajo condiciones de estrés operativo podrían ser invisibilizados para la red neuronal. En consecuencia, el modelo podría volverse más sensible, incrementando potencialmente la tasa de falsos positivos (clasificando un tráfico normal estresado como una anomalía).

Justificación del Negocio y Fronteras de Decisión: A pesar de este riesgo, el submuestreo no es opcional, sino un requisito arquitectónico. Entrenar un modelo Transformer con la distribución original (99.99% a favor de la clase 200) provocaría un colapso en la función de pérdida (Loss Function), forzando al modelo a memorizar la clase mayoritaria para alcanzar un 99% de "precisión" engañosa (Paradoja de la Precisión). Dado que el objetivo primordial de la solución AIOps es el Análisis de Causa Raíz (RCA) y la prevención de fallos críticos, el negocio prefiere asumir un ligero incremento en falsos positivos (una alerta temprana sobre un sistema estresado que al final no falló) antes que permitir un falso negativo (ignorar una caída del Gateway porque el modelo estaba sesgado a predecir siempre "Éxito"). El balanceo perfecto (50/50) reajusta la frontera de decisión del algoritmo, priorizando la sensibilidad (Recall) sobre las clases minoritarias críticas (500, 503, 504).

Reducción de Dimensionalidad (Feature Selection): Finalmente, con el objetivo de consolidar un corpus estructurado y computacionalmente eficiente, se redujo la dimensionalidad del dataset original, pasando de 32 características a únicamente 5 campos esenciales (timestamp, operation, URL, http_status_code, Log_Message_Clean). Este subconjunto altamente refinado constituirá la base exclusiva para la fase de entrenamiento y fine-tuning del modelo predictivo.

Figura 9

Pipeline Automatizado para preparar los datos

Nota: El Pipeline automatizado ha procesado exitosamente la telemetría, enmascarado los textos profundos y conservado el balanceo estadístico (50% tráfico normal, 50% anomalías críticas) preparado en la fase previa.

7. Análisis Exploratorio de Texto (NLP EDA) y Definición de Hiperparámetros

Para la preparación del modelado con arquitecturas Transformer (BERT), se realizó un análisis exploratorio específico sobre la naturaleza del texto no estructurado resultante. Se extrajo la métrica de cantidad de palabras por cada registro de log preprocesado. Los resultados evidencian un corpus altamente conciso y estructurado, con una media de 9.49 palabras por log. Esta baja dispersión es característica de los logs generados por máquinas (Machine-Generated Text), los cuales carecen de la verbosidad típica del lenguaje humano conversacional.

Al evaluar la gráfica (figura 8) de "Distribución de la Longitud de los Logs Enmascarados", se observa un comportamiento multimodal fuertemente marcado, con picos discretos en las 8, 10, 12 y 13 palabras. El pico predominante en las 8 palabras agrupa a la clase mayoritaria (HTTP 200 - Tráfico Normal), cuya plantilla estructurada es más breve. Los

picos secundarios corresponden a las trazas de error inyectadas (HTTP 401, 403, 500, 504), cuyas excepciones requieren secuencias ligeramente más extensas.

Este hallazgo analítico tiene un impacto resolutivo en la fase de ingeniería del modelo. La arquitectura BERT tradicionalmente utiliza un hiperparámetro de longitud máxima de secuencia (max_length) de 512 tokens. Al demostrar estadísticamente que ningún registro supera las 13 palabras, se determina que el modelo se entrenará utilizando un max_length acotado (ej. 32). Esta decisión basada en datos evitará el relleno innecesario de ceros (*padding*), optimizando exponencialmente el uso de memoria VRAM de las GPUs y acelerando los tiempos de cómputo del algoritmo.

Glosario de Conceptos Clave Aplicados:

- *Chunking (Lectura por bloques / Chunksize)*: Técnica de gestión de memoria que consiste en fragmentar archivos masivos (gigabytes) en pequeños trozos (ej. 100,000 líneas). Permite que la infraestructura procese conjuntos masivos de datos (Big Data) sin sufrir desbordamientos de memoria (Out of Memory).
- *JSON (JavaScript Object Notation)*: Formato de texto ligero utilizado para el intercambio de datos. En este proyecto, se empleó para gestionar las claves criptográficas de autenticación seguras (kaggle.json) permitiendo la conexión de la API con los repositorios de datos.
- *Fault Injection (Inyección de Fallos)*: Práctica avanzada de ingeniería de resiliencia orientada a introducir deliberadamente errores o latencias en un sistema (en este caso, de manera sintética en los datos históricos). Esto permite dotar al modelo de Machine Learning con ejemplos de escenarios catastróficos raros o faltantes para prepararlo ante "cisnes negros" operativos.
- *Undersampling (Submuestreo Estratégico)*: Técnica de preprocesamiento estadístico que consiste en reducir aleatoriamente el tamaño de la clase mayoritaria (tráfico normal) para igualar el tamaño de la clase minoritaria (anomalías). Es vital en

proyectos de AIOps para forzar a la red neuronal a prestar atención a los incidentes en lugar de asumir que "todo siempre está bien".

- *Inyección Semántica*: Técnica de enriquecimiento de datos donde se dota de contexto profundo a una métrica estructurada. En este notebook, consistió en traducir un simple error "504" en una traza de texto compleja que incluye variables de telemetría real, simulando la verbosidad de los sistemas en producción.
- *Expresiones Regulares (Regex)*: Secuencias de caracteres que conforman un patrón de búsqueda avanzado. Se utilizaron para localizar de manera eficiente y escalable fragmentos de texto variables (como direcciones IP, milisegundos fluctuantes o IDs numéricos) dentro de cadenas de texto largas.
- *Enmascaramiento de Datos (Data Masking / Anonimización)*: Práctica de ciberseguridad indispensable en ingeniería de datos que consiste en ocultar o reemplazar información sensible (PII - Personally Identifiable Information). Protege la privacidad del usuario y obliga al modelo de Machine Learning a generalizar el contexto del error en lugar de asociarlo a un usuario o sesión específica.
- *Determinismo vs. Estocasticidad (Manipulación de DataFrames)*: Diferenciación metodológica al consultar datos. El uso de métodos estocásticos (como .sample()) permite extraer muestras aleatorias para validar la variabilidad y robustez del código de limpieza, mientras que el uso de métodos deterministas (como .iloc[ ]) garantiza la reproducibilidad exacta al apuntar a un índice estático en la memoria.

Conclusión

El análisis exploratorio y la ingeniería de datos ejecutados a lo largo de este estudio demuestran que la preparación de un corpus para tareas de AIOps requiere intervenciones que trascienden la simple limpieza tabular. La manipulación de un conjunto inicial asimétrico, donde el tráfico exitoso representaba el 99.99% de las instancias, expuso la necesidad de aplicar técnicas de ingeniería de resiliencia, tales como la Inyección de Fallos y el submuestreo estratégico (undersampling). Estas metodologías permitieron forjar un dataset final matemáticamente equilibrado y libre de sesgos frente a la clase mayoritaria, garantizando que el modelo predictivo posea la sensibilidad estadística necesaria para detectar las cinco categorías de incidentes críticos definidos en el alcance.

Adicionalmente, el procesamiento de texto no estructurado mediante la inyección semántica y la normalización con expresiones regulares probó ser un mecanismo efectivo para cumplir con los estándares de ciberseguridad y enmascaramiento de PII (Personally Identifiable Information). Más allá de la protección de datos, la evaluación de la densidad del texto (Word Count) arrojó un hallazgo arquitectónico crucial: la limitación intrínseca de la verbosidad de los logs generados por máquinas. Este descubrimiento permite acotar el hiperparámetro de longitud máxima de secuencia al momento de la tokenización, lo cual se traducirá en una reducción drástica del esfuerzo computacional y del consumo de memoria en la GPU durante la fase de modelado.

En definitiva, el pipeline implementado con los dos notebooks culmina en la consolidación de un Corpus Híbrido Transaccional íntegro y determinista. Las bases analíticas y los parámetros de dimensionalidad aquí establecidos otorgan total viabilidad técnica para proceder a la siguiente fase del proyecto: el entrenamiento, validación y despliegue del modelo predictivo basado en la arquitectura de Transformers, enfocado en la anticipación y diagnóstico de incidentes en pasarelas transaccionales.

Referencias

Buda, M., Maki, A., & Mazurowski, M. A. (2018). A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks, 106*, 249–259. https://doi.org/10.1016/j.neunet.2018.07.011

Guo, H., Yuan, S., & Wu, X. (2021). LogBERT: Log anomaly detection via BERT. *Proceedings of the International Joint Conference on Neural Networks (IJCNN 2021)*, 1–8. https://doi.org/10.1109/IJCNN52387.2021.9534113

Natella, R., Cotroneo, D., & Madeira, H. S. (2016). Assessing dependability with software fault injection: A survey. *ACM Computing Surveys, 48*(3), 1–55. https://doi.org/10.1145/2841425